

HERMÈS

Implémentation

Patterns de code, snippets avant/après
et checklists de validation pour chaque risque
P0 et P1

Ce volume fait suite au rapport d'audit stratégique. Pour chacun des huit risques prioritaires, il fournit le code corrigé prêt à intégrer, le pattern de refactor Avant / Après, les helpers réutilisables, les pièges connus et une checklist de validation. L'objectif est de passer de la recommandation à l'exécution sans ambiguïté de mise en œuvre.

8

RISQUES TRAITÉS

24

SNIPPETS CODE

12

CHECKLISTS

10

HELPERS RÉUTILISABLES

CIBLE

Next.js 16 • React 19 • Prisma 6 • TypeScript 5

PÉRIMÈTRE

src/lib • src/app/api • middleware • prisma • CI

DATE

29 juin 2026

PRÉREQUIS

Lecture préalable du Volume 1 — Audit

HERMÈS • VOLUME 2 —
IMPLÉMENTATION

CONFIDENTIEL • DIFFUSION
RESTREINTE

Table des matières

1. Synthèse exécutive du Volume 2	4
1.1 Périmètre traité	4
1.2 Prérequis et conventions	5
2. R-001 — Réécrire l'authentification	6
2.1 Objectif	6
2.2 Code actuel problématique	6
2.3 Code corrigé	7
2.4 Helper — hashPassword et verifyPassword	8
2.5 Helper — requireSession	9
2.6 Protection anti-brute-force	9
2.7 Checklist de validation	10
2.8 Pièges à éviter	10
3. R-002 — Imposer le multi-tenant	12
3.1 Objectif	12
3.2 Code actuel problématique	12
3.3 Code corrigé	12
3.4 Helper — requireUser	13
3.5 Helper — tenantGuard pour les lectures	13
3.6 Migration des routes existantes	14
3.7 Test d'intrusion multi-tenant	15
3.8 Checklist de validation	15
3.9 Pièges à éviter	16
4. R-003 — Désactiver ignoreBuildErrors	17
4.1 Objectif	17
4.2 Code actuel problématique	17
4.3 Code corrigé	17
4.4 Helper — narrowUnknown	18
4.5 Workflow de correction des 87 any	19
4.6 CI — Gate TypeScript	20

4.7 Checklist de validation	20
4.8 Pièges à éviter	21
5. R-004 & R-005 — Aligner schéma et logs Prisma	22
5.1 Objectif	22
5.2 Code actuel problématique	22
5.3 Migration versionnée	22
5.4 Helper — prismaLog	23
5.5 Seed unique au déploiement	23
5.6 Migration future vers PostgreSQL	24
5.7 Checklist de validation	25
5.8 Pièges à éviter	25
6. R-007 — Rate-limit distribué	26
6.1 Objectif	26
6.2 Code actuel problématique	26
6.3 Helper — rateLimiters par catégorie	26
6.4 Wrapper — applyRateLimit	27
6.5 Tableau des seuils par catégorie	28
6.6 Fallback en cas d'indisponibilité Redis	29
6.7 Checklist de validation	29
6.8 Pièges à éviter	30
7. R-008 — Gestion d'erreurs API unifiée	31
7.1 Objectif	31
7.2 Code actuel problématique	31
7.3 Helper — ApiError	31
7.4 Helper — withErrorHandler	32
7.5 Utilisation type	34
7.6 Format d'erreur normalisé	35
7.7 Checklist de validation	35
7.8 Pièges à éviter	35
8. R-009 — Stratégie de test en trois couches	37
8.1 Objectif	37
8.2 Configuration Vitest	37

8.3 Setup de test — base SQLite in-memory	37
8.4 Helper — createTestUser et createTestSession	38
8.5 Exemple de test unitaire	39
8.6 Configuration Playwright E2E	40
8.7 Parcours E2E prioritaires	41
8.8 Checklist de validation	41
8.9 Pièges à éviter	42
9. R-010 — Headers de sécurité HTTP	43
9.1 Objectif	43
9.2 Code actuel problématique	43
9.3 Helper — securityHeaders	43
9.4 Intégration dans le middleware	45
9.5 Test des en-têtes	46
9.6 Checklist de validation	46
9.7 Pièges à éviter	46
10. Quick wins P2 — Optimisations secondaires	48
10.1 Optimisation des images	48
10.2 Unifier les fichiers de déploiement	48
10.3 Logger structuré	49
10.4 Variables d'environnement typées	50
10.5 Lint rules personnalisés	51
10.6 Checklist globale P2	52
11. Conclusion et prochaines étapes	53
11.1 Prochaines étapes recommandées	53
11.2 Métriques de succès	53
11.3 Liens avec le Volume 3	54

1. Synthèse exécutive du Volume 2

Le Volume 1 a cartographié dix-neuf risques répartis en trois criticités et proposé une roadmap de mise en œuvre sur douze semaines. Le présent volume est le pendant opérationnel : il transforme chaque recommandation stratégique en un ensemble de livrables techniques directement actionnables — snippets de code, helpers réutilisables, arbres de décision, checklists de validation et listes de pièges connus. L'objectif est qu'un développeur prenant en charge un risque P0 ou P1 puisse livrer la correction sans avoir à reconcevoir la solution, et sans risque de réintroduire un défaut déjà identifié lors de l'audit.

Le périmètre couvre les huit risques les plus prioritaires : les trois P0 (authentification, multi-tenant, build strict) et les cinq P1 sélectionnés pour leur impact transverse (cohérence base de données, rate-limit distribué, gestion d'erreurs API, stratégie de test, en-têtes de sécurité). Un dernier chapitre regroupe les quick wins P2 exécutables en moins d'une demi-journée chacun. Les neuf risques P2 restants seront traités dans un Volume 3 dédié aux optimisations de second ordre.

Convention de lecture

Chaque chapitre suit la même structure : **Objectif** (ce que la correction doit accomplir), **Avant** (code problématique extrait du codebase actuel), **Après** (code corrigé prêt à intégrer), **Helper réutilisable** (fonction utilitaire à déposer dans `src/lib/`), **Checklist** (validation avant fusion) et **Pièges à éviter** (retours d'expérience). Les snippets sont en TypeScript et compatibles Next.js 16 / React 19 / Prisma 6.

1.1 Périmètre traité

Risque	Priorité	Chapitre	Helpers produits	Effort
R-001	P0	2	requireSession, hashPassword	3 j
R-002	P0	3	requireUser, tenantGuard	5 j
R-003	P0	4	narrowUnknown, tsc-check	4 j
R-004/005	P1	5	migrate.sh, prismaLog	1 j
R-007	P1	6	rateLimit, rateLimiters	2 j
R-008	P1	7	withErrorHandler, ApiError	2 j
R-009	P1	8	vitest.config, e2e harness	5 j
R-010	P1	9	securityHeaders, csp	1 j
P2 quick wins	P2	10	imageDomains, seedOnce	2 j

Tableau 1 — Cartographie des chapitres du Volume 2 avec efforts estimés et helpers livrés.

1.2 Prérequis et conventions

Avant d'entamer l'implémentation, l'équipe doit s'assurer que l'environnement de travail satisfait les prérequis suivants. Tous les snippets supposent une version de Node supérieure ou égale à 20, un package manager unifié (pnpm de préférence, npm sinon), et un accès en écriture au dépôt GitHub du projet. Les dépendances externes (Upstash Redis, Sentry, Argon2) doivent faire l'objet d'une commande d'abonnement préalable et de l'ajout des secrets associés dans le fichier `.env.local` puis dans le coffre-fort de secrets de la plateforme de déploiement.

- ☐ Node.js ≥ 20.11 LTS installé et activé par défaut via nvm
- ☐ pnpm ≥ 9 installé globalement (`npm i -g pnpm`)
- ☐ Compte Upstash Redis créé, URL + Token récupérés
- ☐ Variables d'environnement NEXTAUTH_SECRET et DATABASE_URL définies en local
- ☐ Branche `main` protégée, branche `audit-fixes` créée
- ☐ Pipeline CI GitHub Actions fonctionnelle (lint + build minimum)
- ☐ Accès administrateur au dashboard de déploiement (Render / Netlify / Vercel)

2. R-001 — Réécrire l'authentification

2.1 Objectif

Remplacer le compte démo hardcodé par un système d'authentification réel, sécurisé et configurable. La solution doit supporter à la fois un provider Credentials (email + mot de passe) pour les utilisateurs sans compte OAuth, et un provider GitHub pour faciliter l'onboarding des utilisateurs techniques. Les mots de passe doivent être hachés avec argon2id — la fonction de hachage recommandée par l'OWASP depuis 2023 — et le secret JWT ne doit plus jamais avoir de valeur de repli hardcodée. L'authentification doit rester active en développement pour éviter les divergences de comportement entre environnements.

2.2 Code actuel problématique

src/lib/auth-config.ts · TypeScript

```
// src/lib/auth-config.ts (extrait)
const users = [{
  id: "1",
  email: "demo@hermes.app",
  password: "hermes2024", // ← en clair, jamais haché
}];

export const authOptions: NextAuthOptions = {
  providers: [
    CredentialsProvider({
      credentials: { email: {}, password: {} },
      async authorize(email, password) {
        const user = users.find(u => u.email === email);
        // ← comparaison en clair, sensible à la timing-attack
        if (user && user.password === password) return user;
        return null;
      },
    }),
  ],
  // ← fallback hardcodé : tokens forgeables si NEXTAUTH_SECRET absent
  secret: process.env.NEXTAUTH_SECRET || "hermes-dev-secret-change-in-production",
};
```

Trois défauts critiques sont visibles dans cet extrait. Premièrement, la liste des utilisateurs est hardcodée dans le source : aucun mécanisme ne permet d'en ajouter un nouveau sans redéployer l'application. Deuxièmement, la comparaison `user.password === password` est sensible aux attaques temporelles (timing attack) — un attaquant peut deviner le mot de passe caractère par caractère en mesurant le temps de réponse. Troisièmement, la valeur de repli pour `NEXTAUTH_SECRET` est publique dans le code source, ce qui permet à quiconque consulte le repo de forger des sessions valides.

2.3 Code corrigé

src/lib/auth-config.ts (1/2) · TypeScript

```
// src/lib/auth-config.ts (partie 1/2 – providers)
import type { NextAuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";
import GitHubProvider from "next-auth/providers/github";
import argon2 from "argon2";
import { prisma } from "@lib/db";

const REQUIRED = (key: string): string => {
  const v = process.env[key];
  if (!v) throw new Error(`Variable d'environnement manquante: ${key}`);
  return v;
};

export const authOptions: NextAuthOptions = {
  secret: REQUIRED("NEXTAUTH_SECRET"), // pas de fallback
  session: { strategy: "jwt", maxAge: 30 * 24 * 60 * 60 }, // 30 j
  providers: [
    GitHubProvider({
      clientId: REQUIRED("GITHUB_CLIENT_ID"),
      clientSecret: REQUIRED("GITHUB_CLIENT_SECRET"),
    }),
    CredentialsProvider({
      credentials: {
        email: { type: "email" },
        password: { type: "password" },
      },
      async authorize(creds) {
        if (!creds?.email || !creds?.password) return null;
        const user = await prisma.user.findUnique({
          where: { email: creds.email.toLowerCase() },
        });
        if (!user || !user.passwordHash) return null;
        const ok = await argon2.verify(user.passwordHash, creds.password);
        return ok
          ? { id: user.id, email: user.email, name: user.name }
          : null;
      },
    }),
  ],
  pages: { signIn: "/login" },
};
```



```
// src/lib/auth-config.ts (partie 2/2 – callbacks)
// (à ajouter dans authOptions du fichier précédent)
callbacks: {
  async jwt({ token, user }) {
    // user n'est défini qu'au premier login ; ensuite on lit le token
    if (user) token.uid = user.id;
    return token;
  },
  async session({ session, token }) {
    // Propage l'uid du token vers la session côté client
    if (session.user) session.user.id = token.uid as string;
    return session;
  },
  async signIn({ user, email }) {
    // Liste blanche de domaine pour le provider GitHub
    if (email?.verificationRequest) return true;
    const allowed = process.env.ALLOWED_EMAIL_DOMAINS?.split(",") ?? [];
    if (allowed.length === 0) return true; // pas de restriction
    const domain = user.email?.split("@")[1];
    return allowed.includes(domain ?? "");
  },
}
}
```

2.4 Helper — hashPassword et verifyPassword

Centraliser le hachage dans un helper dédié garantit que toute nouvelle route d'inscription ou de réinitialisation de mot de passe utilise les mêmes paramètres argon2id. Les valeurs ci-dessous correspondent aux recommandations OWASP 2024 (mémoire 19 MiB, parallélisme 1, itérations 2).

```
// src/lib/password.ts
import argon2 from "argon2";

const OPTIONS: argon2.Options = {
  type: argon2.argon2id,
  memoryCost: 19 * 1024, // 19 MiB
  timeCost: 2, // 2 itérations
  parallelism: 1,
};

export async function hashPassword(plain: string): Promise<string> {
  if (plain.length < 8) throw new Error("Mot de passe trop court (min 8)");
  return argon2.hash(plain, OPTIONS);
}

export async function verifyPassword(
  hash: string,
  plain: string,
): Promise<boolean> {
  try { return await argon2.verify(hash, plain); }
  catch { return false; } // hash malformé → refus silencieux
}
```

2.5 Helper — requireSession

Toutes les routes API doivent récupérer la session via un point d'entrée unique. Ce helper lève une `ApiError` 401 si la session est absente, et expose directement l'identifiant utilisateur — ce qui évite les oublis de vérification dans les routes individuelles.

`src/lib/session.ts` · TypeScript

```
// src/lib/session.ts
import { getServerSession } from "next-auth";
import { authOptions } from "@lib/auth-config";
import { ApiError } from "@lib/api-error";

export async function requireSession() {
  const session = await getServerSession(authOptions);
  if (!session?.user?.id) {
    throw new ApiError(401, "UNAUTHORIZED", "Session invalide ou expirée");
  }
  return session as {
    user: { id: string; email: string; name?: string | null };
  };
}
```

2.6 Protection anti-brute-force

Le rate-limit global de 60 req/min est insuffisant pour la route `/api/auth/callback/credentials`. Un attaquant peut tester 60 mots de passe par minute par IP, soit 86 400 par jour. La protection ci-dessous, à déposer dans `src/lib/auth-rate-limit.ts`, utilise Upstash Redis pour appliquer un seuil de 5 tentatives par 15 minutes par combinaison IP + email, et verrouille le compte pendant 30 minutes après le 5ème échec.

```
// src/lib/auth-rate-limit.ts
import { Ratelimit } from "@upstash/ratelimit";
import { Redis } from "@upstash/redis";
import { ApiError } from "@lib/api-error";

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_URL!,
  token: process.env.UPSTASH_REDIS_TOKEN!,
});

const limiter = new Ratelimit({
  redis,
  limiter: Ratelimit.slidingWindow(5, "15 m"),
  prefix: "auth:credentials",
});

export async function enforceAuthRateLimit(ip: string, email: string) {
  const key = `${ip}:${email.toLowerCase()}`;
  const { success, reset, remaining } = await limiter.limit(key);
  if (!success) {
    const retryAfter = Math.ceil((reset - Date.now()) / 1000);
    throw new ApiError(429, "RATE_LIMITED", "Trop de tentatives", {
      retryAfter,
    });
  }
  return { remaining };
}
```

2.7 Checklist de validation

- ☐ Le compte `demo@hermes.app` est supprimé du code source
- ☐ `NEXTAUTH_SECRET` est défini dans tous les environnements (local, staging, prod)
- ☐ Aucune valeur de repli hardcodée — l'app crash au boot si le secret manque
- ☐ Tous les mots de passe existants en base sont re-hachés via `hashPassword()`
- ☐ La route `/api/auth/callback/credentials` appelle `enforceAuthRateLimit()` avant `authorize()`
- ☐ Le middleware n'a plus de branche `if (process.env.NODE_ENV === 'development')` `return`
- ☐ Le provider GitHub fonctionne en local via `localhost:3000`
- ☐ Les tests E2E couvrent login réussi, login échoué, et verrouillage après 5 tentatives

2.8 Pièges à éviter

Piège à éviter

Ne pas utiliser `argon2` mais `bcrypt` par habitude. Bien que `bcrypt` soit encore acceptable, `argon2id` est désormais le standard recommandé et résiste mieux aux attaques GPU. De plus, `bcrypt` limite les mots de passe à 72 octets, ce qui peut tronquer silencieusement les passphrases.

Piège à éviter

Stocker le hachage dans un champ nommé `password` plutôt que `passwordHash`. Le nom du champ induit en erreur les contributeurs et peut mener à une fuite accidentelle si une route API sérialise l'objet `User` complet par défaut Prisma.

Piège à éviter

Activer le provider GitHub sans restreindre les emails autorisés. Sans liste blanche de domaine ou vérification d'invitation, n'importe quel utilisateur GitHub peut créer un compte. Ajouter un check dans le callback `signIn` : `if (!email.endsWith('@mondomaine.com')) return false.`

3. R-002 — Imposer le multi-tenant

3.1 Objectif

Éliminer la constante `DEFAULT_USER_ID = "default"` utilisée aujourd'hui par l'ensemble des routes API pour écrire et lire des données. Chaque requête doit dorénavant déduire l'identifiant utilisateur de la session authentifiée, et toute requête Prisma doit inclure un filtre `where: { userId }` garantissant l'isolation des données entre locataires. La correction doit être invisible pour le frontend — seul le contrat API change structurellement (chaque ressource est désormais scopée à un utilisateur).

3.2 Code actuel problématique

src/lib/db.ts + leads/route.ts · TypeScript

```
// src/lib/db.ts (extrait problématique)
export const DEFAULT_USER_ID = "default";

export async function ensureDefaultUser() {
  const exists = await prisma.user.findUnique({
    where: { id: "default" },
  });
  if (!exists) {
    await prisma.user.create({
      data: { id: "default", email: "default@hermes.app" },
    });
  }
}

// src/app/api/data/leads/route.ts
import { DEFAULT_USER_ID, ensureDefaultUser } from "@lib/db";

export async function POST(req: Request) {
  await ensureDefaultUser();
  const body = await req.json();
  const lead = await prisma.lead.create({
    data: { ...body, userId: DEFAULT_USER_ID }, // ← partagé entre tous
  });
  return Response.json(lead);
}
```

Le défaut est double. D'une part, `DEFAULT_USER_ID` rend l'isolation entre utilisateurs impossible : toutes les données atterrissent dans le même bac à sable. D'autre part, `ensureDefaultUser()` est appelée avant chaque écriture, ce qui exécute une requête SQL supplémentaire par appel API — un gaspillage de ressources significatif à l'échelle.

3.3 Code corrigé

```
// src/lib/db.ts (réécrit)
import { PrismaClient } from "@prisma/client";

export const prisma = new PrismaClient({
  log: process.env.NODE_ENV === "development"
    ? ["query", "error", "warn"]
    : ["error", "warn"],
});

// Plus de DEFAULT_USER_ID, plus de ensureDefaultUser().
// Le seeding se fait via prisma/seed.ts exécuté au déploiement.

// src/app/api/data/leads/route.ts (réécrit)
import { requireUser } from "@lib/session";
import { withErrorHandler } from "@lib/api-error";
import { prisma } from "@lib/db";

export const POST = withErrorHandler(async (req: Request) => {
  const userId = await requireUser(); // ← lève 401 si pas de session
  const body = await req.json();

  const lead = await prisma.lead.create({
    data: { ...body, userId }, // ← isolation par utilisateur
  });
  return Response.json(lead, { status: 201 });
});
```

3.4 Helper — requireUser

Ce helper est le pivot central du multi-tenant. Il encapsule l'appel à `requireSession()` et ne renvoie que l'identifiant utilisateur — pour les routes qui n'ont pas besoin de l'email ou du nom. Son usage doit devenir systématique dans chaque handler de route API.

```
// src/lib/session.ts (extension)
import { requireSession } from "@lib/session";

export async function requireUser(): Promise<string> {
  const session = await requireSession();
  return session.user.id;
}
```

3.5 Helper — tenantGuard pour les lectures

Le risque le plus subtil du multi-tenant concerne les lectures par identifiant : `findUnique({ where: { id } })` ne filtre pas par utilisateur. Un utilisateur A qui devine l'UUID d'un lead appartenant à B peut le récupérer. Le helper ci-dessous force la vérification d'appartenance après chaque lecture par identifiant.

```
// src/lib/tenant-guard.ts
import { ApiError } from "@lib/api-error";

/**
 * Vérifie qu'une ressource appartient bien à l'utilisateur courant.
 * À utiliser après tout findUnique par ID.
 */
export function assertOwnership<T extends { userId: string }>(
  resource: T | null,
  userId: string,
): T {
  if (!resource) {
    throw new ApiError(404, "NOT_FOUND", "Ressource introuvable");
  }
  if (resource.userId !== userId) {
    // 404 plutôt que 403 pour ne pas confirmer l'existence de la ressource
    throw new ApiError(404, "NOT_FOUND", "Ressource introuvable");
  }
  return resource;
}

// Utilisation :
// const lead = await prisma.lead.findUnique({ where: { id } });
// assertOwnership(lead, userId);
```

3.6 Migration des routes existantes

Le codebase compte une trentaine de routes API à migrer. Plutôt que de tout réécrire en une fois, adopter une approche incrémentale : une route par pull request, avec un test d'intrusion simple. L'audit a identifié les routes ci-dessous comme prioritaires car elles exposent des données personnelles ou stratégiques.

Route	Ressource	Sensibilité	Difficulté
/api/data/leads	Leads B2B	Critique	Triviale
/api/data/contacts	Contacts email/tél	Critique	Triviale
/api/data/deals	Pipelines commerciaux	Élevée	Triviale
/api/data/email-messages	Corps d'emails	Critique	Moyenne
/api/data/workflows	Workflows & rules	Élevée	Moyenne
/api/linkedin/feed	Posts LinkedIn	Moyenne	Moyenne
/api/data/notifications	Notifications	Faible	Triviale
/api/data/feedback	Feedback rules	Faible	Triviale

3.7 Test d'intrusion multi-tenant

Pour chaque route migrée, ajouter un test automatisé vérifiant l'isolation. Le pattern ci-dessous crée deux utilisateurs, crée une ressource pour l'utilisateur A, puis tente d'y accéder avec la session de B — le test doit échouer avec un 404.

tests/integration/tenant-isolation.test.ts · TypeScript

```
// tests/integration/tenant-isolation.test.ts
import { describe, it, expect } from "vitest";
import { createTestUser, createTestSession } from "../helpers/test-harness";

describe("Isolation multi-tenant", () => {
  it("utilisateur B ne peut pas lire le lead de utilisateur A", async () => {
    const userA = await createTestUser({ email: "a@test.local" });
    const userB = await createTestUser({ email: "b@test.local" });

    // A crée un lead
    const sessionA = await createTestSession(userA);
    const createRes = await fetch(`${API}/api/data/leads`, {
      method: "POST",
      headers: { Cookie: sessionA.cookie },
      body: JSON.stringify({ name: "Lead secret", company: "ACME" }),
    });
    const { id: leadId } = await createRes.json();

    // B tente de le lire
    const sessionB = await createTestSession(userB);
    const readRes = await fetch(`${API}/api/data/leads/${leadId}`, {
      headers: { Cookie: sessionB.cookie },
    });

    expect(readRes.status).toBe(404);
  });
});
```

3.8 Checklist de validation

- ☐ La constante `DEFAULT_USER_ID` n'existe plus dans le code
- ☐ La fonction `ensureDefaultUser()` est supprimée (remplacée par `prisma db seed`)
- ☐ Toutes les routes API appellent `requireUser()` en première ligne
- ☐ Tous les `findUnique({ where: { id } })` sont suivis de `assertOwnership()`
- ☐ Le test `tenant-isolation.test.ts` passe pour les 8 routes prioritaires
- ☐ Le schéma Prisma comporte un index `@@index([userId])` sur chaque table métier
- ☐ Aucune route ne renvoie 200 pour une ressource appartenant à un autre utilisateur

3.9 Pièges à éviter

Piège à éviter

Utiliser `findUnique({ where: { id, userId } })` pour vérifier l'appartenance en une seule requête. Bien que séduisante, cette approche masque la distinction entre « ressource inexistante » et « ressource appartenant à autrui ». En cas de bug, le debugging devient impossible. Toujours séparer les deux opérations.

Piège à éviter

Oublier les routes `/api/data/export` et `/api/data/import` lors de la migration. Ces routes batch touchent l'intégralité des données utilisateur et sont parfaites pour exfiltrer massivement. Elles doivent impérativement filtrer par `userId` dans la clause `where` de l'export.

Piège à éviter

Conserver un cache in-memory dans un moteur (workflow-engine, notification-engine) sans clé `userId`. Si le cache est global, un utilisateur A peut récupérer les données de B via le cache. Toujours préfixer la clé de cache par `userId` ou supprimer le cache.

4. R-003 — Désactiver ignoreBuildErrors

4.1 Objectif

Supprimer la directive `typescript: { ignoreBuildErrors: true }` de `next.config.ts`. Cette directive produit un bundle de production même lorsque le compilateur TypeScript détecte des erreurs de types — ce qui masque des bugs silencieux. L'opération s'effectue en deux temps : corriger les erreurs existantes, puis ajouter une étape CI qui empêche toute régression. L'objectif secondaire est de réduire le nombre de `any` (87 occurrences sur 30 fichiers) en les remplaçant par des types explicites ou `unknown` avec narrowing.

4.2 Code actuel problématique

`next.config.ts` · TypeScript

```
// next.config.ts (extrait)
const nextConfig = {
  typescript: {
    ignoreBuildErrors: true, // ← masque toutes les erreurs TS
  },
  eslint: {
    ignoreDuringBuilds: true, // ← masque aussi les warnings ESLint
  },
  images: {
    unoptimized: true, // ← traité chapitre 10
  },
};
```

4.3 Code corrigé

```
// next.config.ts (réécrit)
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  // Plus de ignoreBuildErrors : le build échoue si tsc ou eslint
  // détecte une erreur. C'est le comportement attendu.
  reactStrictMode: true,
  poweredByHeader: false,
  images: {
    formats: ["image/avif", "image/webp"],
    remotePatterns: [
      { protocol: "https", hostname: "lh3.googleusercontent.com" },
      { protocol: "https", hostname: "avatars.githubusercontent.com" },
      { protocol: "https", hostname: "media.licdn.com" },
    ],
  },
  // Headers de sécurité globaux (complétés par middleware.ts)
  async headers() {
    return [{
      source: "/(.*)",
      headers: [
        { key: "X-Content-Type-Options", value: "nosniff" },
        { key: "X-Frame-Options", value: "DENY" },
      ],
    }];
  },
};

export default nextConfig;
```

4.4 Helper — narrowUnknown

Le remplacement des `any` par `unknown` oblige à valider le type avant usage. Le helper `narrowUnknown()` centralise ce pattern via une fonction de garde réutilisable. Il remplace avantageusement les casts `as any` ou `as unknown as Type` qui sont des portes d'entrée pour des bugs silencieux.

```
// src/lib/type-guards.ts
import { ZodSchema, ZodError } from "zod";

/**
 * Valide une valeur inconnue contre un schéma Zod.
 * Lance une ApiError 400 si la validation échoue.
 */
export function narrowUnknown<T>(  
  value: unknown,  
  schema: ZodSchema<T>,  
  label = "payload",  
) : T {  
  const result = schema.safeParse(value);  
  if (!result.success) {  
    const issues = result.error.issues  
      .map((i) => `${i.path.join(".")}: ${i.message}`)  
      .join("; ");  
    throw new Error(`Validation ${label} failed: ${issues}`);  
  }  
  return result.data;  
}  
  
// Utilisation :  
// import { z } from "zod";  
// const LeadSchema = z.object({  
//   name: z.string().min(1),  
//   company: z.string().optional(),  
//   email: z.string().email().optional(),  
// });  
// const lead = narrowUnknown(body, LeadSchema, "lead");
```

4.5 Workflow de correction des 87 any

L'audit a identifié 87 occurrences de **any** réparties sur 30 fichiers. Plutôt que de tout corriger en une fois, adopter une stratégie par paliers : traiter d'abord les fichiers `src/lib` (impact runtime direct), ensuite les routes API, enfin les composants UI. Pour chaque fichier, utiliser la commande `rg "\\bany\\b" --type ts` pour lister les occurrences, puis remplacer selon l'arbre de décision ci-dessous.

Type de any	Fréquence	Stratégie	Effort unitaire
any sur params de fonction	32%	Définir un type dédié ou unknown + Zod	15 min
any sur retour de fetch()	24%	Schéma Zod + narrowUnknown	10 min
any sur callback (Promise, event)	18%	Type générique ou interface callback	8 min
any sur props composant React	14%	Définir Props interface	5 min
any légitime (catch error)	12%	Remplacer par unknown + instanceof	3 min

4.6 CI — Gate TypeScript

Ajouter une étape CI obligatoire qui exécute `tsc --noEmit` sur chaque pull request. Cette étape doit bloquer la fusion si le compilateur détecte une erreur. Combinée à la suppression de `ignoreBuildErrors`, elle garantit qu'aucune régression de type ne peut atteindre la branche principale.

`.github/workflows/ci.yml` · YAML

```
# .github/workflows/ci.yml (extrait)
name: CI
on:
  pull_request:
  branches: [main]
jobs:
  typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
      with: { version: 9 }
      - uses: actions/setup-node@v4
      with:
        node-version: 20
      cache: pnpm
      - run: pnpm install --frozen-lockfile
      - run: pnpm exec tsc --noEmit
      - run: pnpm lint
      - run: pnpm test --coverage
      - name: Couverture minimale
      run: |
        COV=$(pnpm exec c8 report --reporter=text-summary | grep "Lines" | awk '{print $2}' | tr -d
        '%')
        if [ "$COV" -lt 50 ]; then
        echo "Couverture $COV% < 50% – bloqué"
        exit 1
        fi
```

4.7 Checklist de validation

- ☐ La directive `ignoreBuildErrors` est supprimée de `next.config.ts`
- ☐ La directive `ignoreDuringBuilds` (ESLint) est également supprimée
- ☐ `npm tsc --noEmit` ne renvoie aucune erreur
- ☐ `npm next build` réussit sans warning ni error
- ☐ L'étape `typecheck` est active dans la CI et bloque les PR
- ☐ Le nombre de `any` est descendu sous 20 (cible initiale)
- ☐ Aucun nouveau `any` n'est introduit (vérifié par un lint rule custom)

4.8 Pièges à éviter

Piège à éviter

Activer `strict: true` dans `tsconfig` d'un seul coup. Le mode strict active 7 vérifications distinctes (`noImplicitAny`, `strictNullChecks`, etc.) qui produiront des centaines d'erreurs d'un coup. Activer les vérifications une par une, dans l'ordre : `noImplicitReturns` → `noFallthroughCasesInSwitch` → `strictNullChecks` → reste.

Piège à éviter

Remplacer un `any` par `unknown` sans ajout de garde. Cela déplace le problème : le code compile, mais le runtime peut casser si la valeur n'a pas la forme attendue. Toujours accompagner `unknown` d'un schéma Zod ou d'une garde explicite.

Piège à éviter

Activer `reactStrictMode: true` sans auditer les `useEffect` existants. Le mode strict double les effets en développement, ce qui peut révéler des effets de bord cachés (double appel API, double inscription WebSocket). Vérifier chaque `useEffect` avant de déployer.

5. R-004 & R-005 — Aligner schéma et logs Prisma

5.1 Objectif

Résoudre deux défauts corrélés : l'incohérence entre le provider PostgreSQL déclaré dans le schéma Prisma et la chaîne SQLite utilisée à l'exécution, et la verbosité excessive des logs Prisma en production (`log: ['query']` génère une ligne par requête SQL). Le chapitre livre également le script de migration versionnée et la procédure de suppression de la fonction `ensureDefaultUser()` déjà traitée au chapitre 3.

5.2 Code actuel problématique

Avant

```
// prisma/schema.prisma
datasource db {
  provider = "postgresql" // ← déclaré PG
  url = env("DATABASE_URL")
}

// src/lib/db.ts
new PrismaClient({
  log: ["query", "error", "warn"], // ←
  log: query en prod
});

// .env
DATABASE_URL="file:./db/custom.db" // ←
SQLite !
```

Après

```
// prisma/schema.prisma
datasource db {
  provider = "sqlite" // ← aligné sur
  DATABASE_URL
  url = env("DATABASE_URL")
}

// src/lib/db.ts
new PrismaClient({
  log: process.env.NODE_ENV === "development"
    ? ["query", "error", "warn"]
    : ["error", "warn"], // ← silence en prod
});

// .env
DATABASE_URL="file:./db/custom.db" // ←
inchangé
```

L'incohérence provider PG / runtime SQLite est silencieuse en développement car Prisma adapte dynamiquement le dialecte. Mais certaines fonctionnalités avancées (JSON indexé, arrays, enums PostgreSQL) échoueront en production si une migration vers PostgreSQL a lieu sans ajustement du schéma. La correction la plus simple à court terme est d'aligner le provider sur SQLite. Une migration vers PostgreSQL est recommandée à moyen terme (chapitre 5.6).

5.3 Migration versionnée

Remplacer l'usage de `prisma db push` (qui ne versionne pas les migrations) par `prisma migrate`. Cette commande génère un fichier SQL horodaté dans `prisma/migrations/` qui peut être rejoué sur n'importe quel environnement. Le script ci-dessous automatise la première migration depuis un schéma existant.

scripts/prisma-migrate-init.sh · Bash

```
#!/usr/bin/env bash
# scripts/prisma-migrate-init.sh
set -euo pipefail

echo "→ Création de la migration initiale..."
pnpm exec prisma migrate dev --name init --create-only

echo "→ Vérification du SQL généré..."
ls -la prisma/migrations/

echo "→ Application de la migration..."
pnpm exec prisma migrate dev

echo "→ Génération du client Prisma..."
pnpm exec prisma generate

echo "→ Seed initial..."
pnpm exec prisma db seed

echo "✓ Migration initiale terminée"
```

5.4 Helper — prismaLog

Plutôt que d'inliner la condition ternaire dans chaque instantiation Prisma, le helper `prismaLog()` centralise la logique et facilite un ajustement ultérieur (par exemple ajouter un niveau `info` en staging).

src/lib/prisma-log.ts · TypeScript

```
// src/lib/prisma-log.ts
import type { PrismaClientOptions } from "@prisma/client";

export function prismaLog(): PrismaClientOptions["log"] {
  if (process.env.NODE_ENV === "development") {
    return ["query", "error", "warn"];
  }
  if (process.env.NODE_ENV === "test") {
    return ["error"]; // silence total en test
  }
  return ["error", "warn"]; // production
}

// Utilisation :
// import { prismaLog } from "@lib/prisma-log";
// export const prisma = new PrismaClient({ log: prismaLog() });
```

5.5 Seed unique au déploiement

Supprimer la fonction `ensureDefaultUser()` appelée à chaque écriture (voir chapitre 3) et la remplacer par un seed exécuté une seule fois au déploiement. Le script ci-dessous crée un utilisateur admin initial à partir de variables d'environnement, et ajoute les règles de feedback par défaut.


```
// prisma/seed.ts
import { PrismaClient } from "@prisma/client";
import { hashPassword } from "../src/lib/password";

const prisma = new PrismaClient();

async function main() {
  const adminEmail = process.env.ADMIN_EMAIL;
  const adminPassword = process.env.ADMIN_PASSWORD;

  if (!adminEmail || !adminPassword) {
    console.warn("ADMIN_EMAIL/ADMIN_PASSWORD manquants – skip seed admin");
  } else {
    const exists = await prisma.user.findUnique({
      where: { email: adminEmail },
    });
    if (!exists) {
      await prisma.user.create({
        data: {
          email: adminEmail,
          name: "Admin",
          passwordHash: await hashPassword(adminPassword),
          role: "ADMIN",
        },
      });
      console.log(`✓ Admin ${adminEmail} créé`);
    }
  }

  // Règles de feedback par défaut
  await prisma.feedbackRule.upsert({
    where: { id: "default-positive" },
    create: { id: "default-positive", pattern: "merci|super|génial", action: "BOOST" },
    update: {},
  });

  console.log("✓ Seed terminé");
}

main()
  .catch((e) => { console.error(e); process.exit(1); })
  .finally(async () => { await prisma.$disconnect(); });
```

5.6 Migration future vers PostgreSQL

À moyen terme (semaines 7 à 9 de la roadmap), il est recommandé de migrer vers PostgreSQL pour bénéficier des fonctionnalités absentes de SQLite : transactions concurrentes, JSON indexé, arrays, recherche full-text, extensions (pg_trgm, PostGIS). La procédure consiste à : (1) provisionner une instance PostgreSQL managée (Neon, Supabase ou RDS), (2) modifier `prisma/schema.prisma` pour repasser le provider à `postgresql`, (3) ajuster les types spécifiques (par exemple `String` → `String @db.Text` pour les corps d'email), (4) générer une nouvelle migration, (5) exécuter un script de transfert de données SQLite → PostgreSQL.

5.7 Checklist de validation

- ☐ Le provider Prisma correspond au schéma de `DATABASE_URL`
- ☐ `log: [ 'query' ]` est désactivé en production
- ☐ `prisma/migrations/` contient au moins une migration versionnée
- ☐ `prisma db push` n'est plus utilisé dans le workflow
- ☐ Le fichier `prisma/seed.ts` est exécutable via `pnpm exec prisma db seed`
- ☐ La commande `prisma db seed` est ajoutée au script de post-déploiement
- ☐ `ensureDefaultUser()` est supprimée du code applicatif
- ☐ Les logs en production ne contiennent plus de lignes `prisma:query`

5.8 Pièges à éviter

Piège à éviter

Oublier de supprimer le fichier `db/custom.db` après migration vers PostgreSQL. Le fichier reste présent mais inutilisé, ce qui peut induire en erreur un développeur pensant que SQLite est encore actif. Le supprimer et ajouter une entrée `db/*.db` dans `.gitignore`.

Piège à éviter

Mixer `prisma db push` et `prisma migrate` dans le workflow. `db push` ne crée pas de migration et peut laisser le schéma en base désynchronisé des migrations versionnées. Standardiser sur `migrate dev` en développement et `migrate deploy` en production.

Piège à éviter

Exécuter le seed en production sans idempotence. Si le script ne vérifie pas l'existence préalable, il échouera sur une contrainte unique au second déploiement. Toujours utiliser `upsert()` ou vérifier avec `findUnique()` avant `create()`.

6. R-007 — Rate-limit distribué

6.1 Objectif

Remplacer le rate-limit in-memory actuel — basé sur `Map<string, ...>` — par un rate-limit distribué s'appuyant sur Upstash Redis. La solution doit fonctionner en multi-instance (Render, Netlify Functions, conteneurs Kubernetes), supporter des seuils différenciés par catégorie de route, et exposer les en-têtes standard `X-RateLimit-*` pour permettre aux clients d'adapter leur cadence.

6.2 Code actuel problématique

src/middleware.ts · TypeScript

```
// src/middleware.ts (extrait)
const rateLimitMap = new Map<string, { count: number; lastReset: number }>();
const RATE_LIMIT_WINDOW = 60_000; // 1 minute
const RATE_LIMIT_MAX = 60;

function rateLimit(ip: string): boolean {
  const now = Date.now();
  const entry = rateLimitMap.get(ip);
  if (!entry || now - entry.lastReset > RATE_LIMIT_WINDOW) {
    rateLimitMap.set(ip, { count: 1, lastReset: now });
    return true;
  }
  entry.count++;
  return entry.count <= RATE_LIMIT_MAX;
}
```

Trois défauts : (1) la Map est privée à chaque instance, donc en multi-instance le seuil effectif est multiplié par le nombre d'instances ; (2) le seuil unique de 60 req/min est trop généreux pour `/api/auth/*` et trop strict pour les routes de lecture agrégées ; (3) la Map grandit indéfiniment (pas de garbage collection des IPs anciennes), ce qui constitue une fuite mémoire à long terme.

6.3 Helper — rateLimiters par catégorie

```
// src/lib/rate-limit.ts
import { Ratelimit } from "@upstash/ratelimit";
import { Redis } from "@upstash/redis";

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_URL!,
  token: process.env.UPSTASH_REDIS_TOKEN!,
});

// Catégories de routes avec seuils différenciés
const buildLimiter = (max: number, window: string, prefix: string) =>
  new Ratelimit({
    redis,
    limiter: Ratelimit.slidingWindow(max, window),
    prefix,
    analytics: true,
  });

export const rateLimiters = {
  auth: buildLimiter(5, "15 m", "rl:auth"), // login, signup
  write: buildLimiter(30, "1 m", "rl:write"), // POST/PUT/DELETE
  read: buildLimiter(120, "1 m", "rl:read"), // GET agrégés
  ai: buildLimiter(10, "1 m", "rl:ai"), // chat, generate
  linkedin: buildLimiter(20, "1 m", "rl:linkedin"), // API LinkedIn
  export: buildLimiter(3, "1 h", "rl:export"), // bulk export
};

export type RateLimitCategory = keyof typeof rateLimiters;

export async function enforceRateLimit(
  category: RateLimitCategory,
  identifiant: string,
): Promise<{ remaining: number; reset: number }> {
  const limiter = rateLimiters[category];
  const { success, remaining, reset } = await limiter.limit(identifiant);
  if (!success) {
    const retryAfter = Math.ceil((reset - Date.now()) / 1000);
    return { remaining: 0, reset: retryAfter };
  }
  return { remaining, reset };
}
```

6.4 Wrapper — applyRateLimit

Plutôt que d'injecter le rate-limit dans chaque route individuellement, le wrapper `applyRateLimit()` s'enroule autour du handler et applique le bon limiteur selon la catégorie. Il injecte également les en-têtes `X-RateLimit-Remaining` et `Retry-After` dans la réponse.

```
// src/lib/rate-limit-wrapper.ts
import { NextRequest, NextResponse } from "next/server";
import { enforceRateLimit, RateLimitCategory } from "@lib/rate-limit";

export function applyRateLimit(
  category: RateLimitCategory,
  handler: (req: NextRequest, ctx: { userId?: string }) => Promise<Response>,
) {
  return async (req: NextRequest): Promise<Response> => {
    const ip = req.headers.get("x-forwarded-for")?.split(",")[0]
    || req.headers.get("x-real-ip")
    || "unknown";

    const { remaining, reset } = await enforceRateLimit(category, ip);

    if (remaining === 0) {
      return NextResponse.json(
        { error: { code: "RATE_LIMITED", message: "Trop de requêtes" } },
        {
          status: 429,
          headers: {
            "Retry-After": String(reset),
            "X-RateLimit-Remaining": "0",
            "X-RateLimit-Limit": String(category),
          },
        },
      );
    }

    const res = await handler(req, {});
    res.headers.set("X-RateLimit-Remaining", String(remaining));
    return res;
  };
}

// Utilisation :
// export const POST = applyRateLimit("write", async (req) => { ... });
// export const GET = applyRateLimit("read", async (req) => { ... });
```

6.5 Tableau des seuils par catégorie

Catégorie	Routes	Limite	Window	Justification
auth	/api/auth/*	5	15 min	Anti-brute-force
write	POST/PUT/DELETE /api/data/*	30	1 min	Charge utile Prisma
read	GET /api/data/*	120	1 min	UI déroule des listes
ai	/api/ai/*	10	1 min	Coût OpenAI/Anthropic
linkedin	/api/linkedin/*	20	1 min	Quota API LinkedIn
export	/api/data/export	3	1 h	Requêtes lourdes

6.6 Fallback en cas d'indisponibilité Redis

Si Upstash Redis est indisponible (panne réseau, quota dépassé), l'application doit continuer à fonctionner en mode dégradé. Le fallback ci-dessous bascule sur un rate-limit local (instance-courante) et logge un warning. Cela garantit que la disponibilité de l'app n'est pas couplée à celle de Redis.

src/lib/rate-limit-fallback.ts · TypeScript

```
// src/lib/rate-limit-fallback.ts
import { Ratelimit } from "@upstash/ratelimit";

// Rate-limit local (instance unique) – fallback si Redis down
const localLimiter = new Ratelimit({
  // @ts-expect-error – Ratelimit supporte un store local
  limiter: Ratelimit.slidingWindow(60, "1 m"),
  prefix: "rl:local",
  analytics: false,
});

export async function safeRateLimit(
  category: string,
  ip: string,
): Promise<{ success: boolean; remaining: number; reset: number }> {
  try {
    // Tentative Redis d'abord
    const { enforceRateLimit } = await import("@/lib/rate-limit");
    const r = await enforceRateLimit(category as any, ip);
    return { success: r.remaining > 0, ...r };
  } catch (e) {
    console.warn("[rate-limit] Redis down, fallback local:", e);
    const r = await localLimiter.limit(`${category}:${ip}`);
    return { success: r.success, remaining: r.remaining, reset: r.reset };
  }
}
```

6.7 Checklist de validation

- ☐ Upstash Redis est provisionné (URL + Token dans .env)
- ☐ La Map `rateLimitMap` est supprimée du middleware
- ☐ Chaque route API est wrappée par `applyRateLimit()`
- ☐ Les en-têtes `X-RateLimit-Remaining` sont présents dans les réponses
- ☐ Un test simule 100 appels en rafale sur `/api/auth/*` et vérifie le 429
- ☐ Le fallback local fonctionne quand Redis est volontairement coupé
- ☐ Le dashboard Upstash affiche l'utilisation réelle des limiteurs

6.8 Pièges à éviter

Piège à éviter

Utiliser l'IP brute comme identifiant de rate-limit. Plusieurs utilisateurs légitimes peuvent partager une IP (NAT d'entreprise, VPN, mobile). Combiner IP + identifiant utilisateur authentifié pour éviter les faux positifs :

```
`${ip}:${userId ?? 'anon'}`.
```

Piège à éviter

Oublier de configurer `UPSTASH_REDIS_URL` et `UPSTASH_REDIS_TOKEN` en production. Sans ces variables, le rate-limit plantera silencieusement (sauf si le fallback du chapitre 6.6 est en place). Toujours tester le démarrage avec Redis down pour valider le fallback.

Piège à éviter

Choisir le mauvais algorithme. `fixedWindow` autorise 2× la limite aux frontières de fenêtre (60 req à 11:59 + 60 req à 12:00 = 120 req en 1 minute). Préférer `slidingWindow` qui lisse les transitions.

7. R-008 — Gestion d'erreurs API unifiée

7.1 Objectif

Créer un wrapper `withErrorHandler()` qui emballe systématiquement le handler de chaque route API. Le wrapper capture les erreurs Prisma connues et renvoie un format JSON normalisé `{ error: { code, message, details? } }`, avec les codes HTTP appropriés. Il logge les erreurs 500 avec un identifiant de corrélation et ne divulgue jamais le message Prisma brut en production.

7.2 Code actuel problématique

src/app/api/data/leads/route.ts · TypeScript

```
// src/app/api/data/leads/route.ts (actuel)
export async function POST(req: Request) {
  const body = await req.json();
  const lead = await prisma.lead.create({
    data: { ...body, userId: DEFAULT_USER_ID },
  });
  return Response.json(lead);
}

// Si body.email est en doublon → Prisma lève P2002
// → Next.js renvoie 500 avec le message Prisma en clair
// → le frontend ne sait pas distinguer 400 (client) vs 500 (serveur)
```

7.3 Helper — `ApiError`

La classe `ApiError` est le type d'erreur canonique du projet. Toute erreur métier ou technique doit être levée comme `ApiError`, ce qui permet au wrapper de la convertir en réponse HTTP structurée sans logique de mapping dispersée dans les routes.


```
// src/lib/api-error.ts
export class ApiError extends Error {
  constructor(
    public status: number,
    public code: string,
    message: string,
    public details?: Record<string, unknown>,
  ) {
    super(message);
    this.name = "ApiError";
  }

  static unauthorized(msg = "Non authentifié") {
    return new ApiError(401, "UNAUTHORIZED", msg);
  }
  static forbidden(msg = "Action interdite") {
    return new ApiError(403, "FORBIDDEN", msg);
  }
  static notFound(msg = "Ressource introuvable") {
    return new ApiError(404, "NOT_FOUND", msg);
  }
  static conflict(msg = "Conflit", details?: Record<string, unknown>) {
    return new ApiError(409, "CONFLICT", msg, details);
  }
  static validation(msg: string, details?: Record<string, unknown>) {
    return new ApiError(422, "VALIDATION_ERROR", msg, details);
  }
  static rateLimited(retryAfter: number) {
    return new ApiError(429, "RATE_LIMITED", "Trop de requêtes", { retryAfter });
  }
  static internal(msg = "Erreur interne") {
    return new ApiError(500, "INTERNAL_ERROR", msg);
  }
}
```

7.4 Helper — withErrorHandler

Le wrapper `withErrorHandler()` capture les erreurs et les convertit en réponses HTTP. Il gère trois familles : (1) `ApiError` levée explicitement par le code métier ; (2) erreurs Prisma reconnues (P2002, P2025, etc.) ; (3) autres erreurs (500 générique avec identifiant de corrélation).

```

// src/lib/with-error-handler.ts (partie 1/2)
import { Prisma } from "@prisma/client";
import { ApiError } from "@lib/api-error";
import { logger } from "@lib/logger";
import { randomUUID } from "crypto";

type Handler<T> = (req: Request, ctx: { params?: Record<string, string> }) => Promise<T>;

export function withErrorHandler<T extends Response>(
  handler: Handler<T>,
): Handler<T> {
  return async (req, ctx) => {
    const correlationId = randomUUID();
    try {
      const res = await handler(req, ctx);
      res.headers.set("X-Correlation-Id", correlationId);
      return res;
    } catch (err) {
      // 1. ApiError explicite
      if (err instanceof ApiError) {
        return Response.json(
          { error: { code: err.code, message: err.message, details: err.details } },
          { status: err.status, headers: { "X-Correlation-Id": correlationId } },
        ) as T;
      }
      // 2. Erreurs Prisma connues
      if (err instanceof Prisma.PrismaClientKnownRequestError) {
        const mapped = mapPrismaError(err);
        return Response.json(
          { error: mapped },
          { status: mapped.status, headers: { "X-Correlation-Id": correlationId } },
        ) as T;
      }
      // 3. Erreur inattendue → 500 générique + log
      logger.error({
        correlationId,
        message: err instanceof Error ? err.message : String(err),
        stack: err instanceof Error ? err.stack : undefined,
        url: req.url, method: req.method,
      });
      const isProd = process.env.NODE_ENV === "production";
      return Response.json(
        { error: {
          code: "INTERNAL_ERROR",
          message: isProd ? "Une erreur est survenue"
            : err instanceof Error ? err.message : String(err),
          correlationId,
        } },
        { status: 500, headers: { "X-Correlation-Id": correlationId } },
      ) as T;
    }
  };
}

```

```
// src/lib/with-error-handler.ts (partie 2/2 – mapPrismaError)
function mapPrismaError(err: Prisma.PrismaClientKnownRequestError) {
  switch (err.code) {
    case "P2002": // Unique constraint
      return {
        status: 409,
        code: "CONFLICT",
        message: "Cette ressource existe déjà",
        details: { target: err.meta?.target },
      };
    case "P2025": // Record not found
      return { status: 404, code: "NOT_FOUND", message: "Ressource introuvable" };
    case "P2003": // Foreign key
      return {
        status: 400,
        code: "FOREIGN_KEY_VIOLATION",
        message: "Référence invalide",
        details: { field: err.meta?.field_name },
      };
    case "P1001": // DB timeout
      return {
        status: 503,
        code: "DATABASE_UNAVAILABLE",
        message: "Base de données temporairement indisponible",
      };
    default:
      return { status: 500, code: "DATABASE_ERROR", message: "Erreur de base de données" };
  }
}
```

7.5 Utilisation type

Avant

```
// AVANT
export async function POST(req: Request) {
  const body = await req.json();
  const lead = await prisma.lead.create({
    data: { ...body, userId: DEFAULT_USER_ID },
  });
  return Response.json(lead);
}
```

Après

```
// APRÈS
import { withErrorHandler } from
"@/lib/with-error-handler";
import { requireUser } from "@/lib/session";
import { ApiError } from "@/lib/api-error";

export const POST = withErrorHandler(async
(req) => {
  const userId = await requireUser();
  const body = await req.json();

  if (!body.name) {
    throw ApiError.validation("Champ name
requis", { field: "name" });
  }

  const lead = await prisma.lead.create({
    data: { ...body, userId },
  });
  return Response.json(lead, { status: 201 });
});
```

7.6 Format d'erreur normalisé

Code HTTP	Code API	Quand l'utiliser	Exemple
400	BAD_REQUEST	Payload malformé	JSON invalide
401	UNAUTHORIZED	Session absente/expirée	Token JWT invalide
403	FORBIDDEN	Action non autorisée	Modifier lead d'autrui
404	NOT_FOUND	Ressource inexistante	Lead UUID inconnu
409	CONFLICT	Contrainte unique violée	Email déjà pris
422	VALIDATION_ERROR	Schéma Zod invalide	Email mal formaté
429	RATE_LIMITED	Quota dépassé	5 logins / 15 min
500	INTERNAL_ERROR	Erreur inattendue	Bug runtime
503	DATABASE_UNAVAILABLE	DB down	Prisma P1001

7.7 Checklist de validation

- ☐ Toutes les routes API sont wrappées par `withErrorHandler()`
- ☐ Aucune route ne renvoie directement le message Prisma en production
- ☐ Chaque réponse contient un `X-Correlation-Id`
- ☐ Les erreurs 500 sont loggées via `logger.error()` avec stack trace
- ☐ Le frontend parse les erreurs via une fonction `parseApiError()` shared
- ☐ Un test vérifie qu'une contrainte unique renvoie bien un 409
- ☐ Un test vérifie qu'une erreur Prisma inconnue renvoie un 500 sans fuite du message

7.8 Pièges à éviter

Piège à éviter

Logguer les erreurs 4xx au même niveau que les 5xx. Les 4xx sont des erreurs client (validation, auth) et ne méritent pas un `logger.error()`. Utiliser `logger.info()` pour les 4xx et réserver `error` aux 5xx. Sinon Sentry/ELK sera saturé de bruit inutile.

Piège à éviter

Renvoyer le `err.stack` dans la réponse HTTP en production. Le stack trace divulgue des informations sur la structure du code (chemins de fichiers, noms de fonctions) qui facilitent les attaques. Toujours vérifier `NODE_ENV === 'development'` avant de l'inclure.

Piège à éviter

Wrapper un handler qui n'utilise pas `Response.json()` ou `NextResponse.json()`. Le wrapper suppose que le handler renvoie un objet `Response` — sinon l'injection du header `X-Correlation-Id` plantera. Toujours typer le handler de retour comme `Promise<Response>`.

8. R-009 — Stratégie de test en trois couches

8.1 Objectif

Mettre en place une stratégie de test couvrant les trois couches complémentaires : unitaire (fonctions pures), intégration (routes API avec base de test) et end-to-end (parcours critiques via Playwright). La cible initiale est 60 % de couverture sur `src/lib`, 40 % sur `src/app/api`, et 5 parcours E2E couvrant les chemins critiques de l'application.

8.2 Configuration Vitest

`vitest.config.ts` · TypeScript

```
// vitest.config.ts
import { defineConfig } from "vitest/config";
import { resolve } from "path";

export default defineConfig({
  test: {
    environment: "node",
    include: ["tests/unit/**/*.test.ts", "tests/integration/**/*.test.ts"],
    exclude: ["tests/e2e/**", "node_modules/**"],
    coverage: {
      provider: "v8",
      reporter: ["text", "html", "lcov"],
      include: ["src/lib/**", "src/app/api/**"],
      exclude: ["**/*.d.ts", "**/index.ts"],
      thresholds: {
        lines: 50,
        functions: 50,
        branches: 40,
        statements: 50,
      },
    },
    setupFiles: [".tests/setup.ts"],
    globalSetup: ".tests/global-setup.ts",
    resolve: {
      alias: { "@": resolve(__dirname, "src") },
    },
  },
});
```

8.3 Setup de test — base SQLite in-memory

Pour les tests d'intégration, utiliser une base SQLite in-memory isolée par test. Le setup ci-dessous crée un schéma frais avant chaque test et le détruit après, garantissant qu'aucun état ne fuite entre les tests.

tests/setup.ts + tests/global-setup.ts · TypeScript

```
// tests/setup.ts
import { execSync } from "child_process";
import { prisma } from "@lib/db";

beforeEach(async () => {
  // Recrée le schéma in-memory pour chaque test
  await prisma.$executeRawUnsafe("DELETE FROM User WHERE id != '__system__'");
  // Plus simple : utiliser prisma migrate reset en début de suite
});

afterAll(async () => {
  await prisma.$disconnect();
});

// tests/global-setup.ts
import { execSync } from "child_process";

export async function setup() {
  process.env.DATABASE_URL = "file:./db/test.db";
  // Reset la DB de test
  execSync("pnpm exec prisma migrate reset --force --skip-seed", {
    stdio: "inherit",
  });
}

export async function teardown() {
  // Nettoyage final
}
```

8.4 Helper — createTestUser et createTestSession

Centraliser la création d'utilisateurs et de sessions de test évite la duplication de code et garantit que tous les tests utilisent des fixtures cohérentes. Le helper ci-dessous crée un utilisateur en base et génère une session JWT valide pour les tests d'intégration.

tests/helpers/test-harness.ts (1/2) · TypeScript

```
// tests/helpers/test-harness.ts (partie 1/2 – createTestUser)
import { prisma } from "@lib/db";
import { hashPassword } from "@lib/password";

let counter = 0;

export async function createTestUser(
  opts: { email?: string; role?: string } = {},
) {
  counter++;
  const email = opts.email ?? `test-${counter}@test.local`;
  const user = await prisma.user.create({
    data: {
      email,
      name: `Test User ${counter}`,
      passwordHash: await hashPassword("TestPassword123!"),
      role: opts.role ?? "USER",
    },
  });
  return user;
}
```

tests/helpers/test-harness.ts (2/2) · TypeScript

```
// tests/helpers/test-harness.ts (partie 2/2 – createTestSession)
import { authOptions } from "@lib/auth-config";

export async function createTestSession(
  user: { id: string; email: string },
) {
  // Génère un token JWT valide via NextAuth
  const token = await (authOptions.callbacks?.jwt as any)?.({
    token: { uid: user.id },
    user: { id: user.id, email: user.email },
    account: null,
    profile: undefined,
    isNewUser: false,
  });

  return {
    cookie: `next-auth.session-token=${token}`,
    token,
  };
}

export const API =
  process.env.TEST_API_URL ?? "http://localhost:3000";
```

8.5 Exemple de test unitaire


```
// tests/unit/password.test.ts
import { describe, it, expect } from "vitest";
import { hashPassword, verifyPassword } from "@lib/password";

describe("password", () => {
  it("hache un mot de passe et le vérifie", async () => {
    const hash = await hashPassword("S3cret!Pass");
    expect(hash).not.toBe("S3cret!Pass");
    expect(hash.startsWith("$argon2id$")).toBe(true);

    const ok = await verifyPassword(hash, "S3cret!Pass");
    expect(ok).toBe(true);

    const ko = await verifyPassword(hash, "wrong");
    expect(ko).toBe(false);
  });

  it("rejette les mots de passe trop courts", async () => {
    await expect(hashPassword("abc")).rejects.toThrow("trop court");
  });

  it("gère un hash malformé sans crasher", async () => {
    const ok = await verifyPassword("not-a-hash", "anything");
    expect(ok).toBe(false);
  });
});
```

8.6 Configuration Playwright E2E

```
// playwright.config.ts
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests/e2e",
  fullyParallel: false, // série : la base de test est partagée
  retries: process.env.CI ? 2 : 0,
  workers: 1,
  reporter: process.env.CI ? "github" : "html",
  use: {
    baseURL: "http://localhost:3000",
    trace: "on-first-retry",
    screenshot: "only-on-failure",
    storageState: undefined, // pas de session persistée
  },
  projects: [
    { name: "chromium", use: { ...devices["Desktop Chrome"] } },
    { name: "mobile", use: { ...devices["iPhone 14"] } },
  ],
  webServer: {
    command: "pnpm dev",
    url: "http://localhost:3000",
    reuseExistingServer: !process.env.CI,
    timeout: 60_000,
  },
});
```

8.7 Parcours E2E prioritaires

#	Parcours	Étapes clés	Assertion
1	Login + accès dashboard	Login, vérif session, dashboard	URL = /dashboard
2	Création de lead	Form, submit, liste refresh	Lead dans la liste
3	Génération post IA	Topic input, générer, prévisualiser	3 suggestions affichées
4	Programmation LinkedIn	Sélection post, date future, schedule	Post en file d'attente
5	Logout + protection route	Logout, tentative /dashboard	Redirigé vers /login

8.8 Checklist de validation

- ❑ `vitest` et `@playwright/test` installés
- ❑ `pnpm test` exécute les tests unitaires et d'intégration
- ❑ `pnpm test:e2e` exécute les parcours Playwright
- ❑ La couverture sur `src/lib` atteint 60 %
- ❑ La couverture sur `src/app/api` atteint 40 %
- ❑ Les 5 parcours E2E passent en CI
- ❑ L'étape `CI test` bloque les PR si la couverture baisse
- ❑ Un badge de couverture est affiché dans le README

8.9 Pièges à éviter

Piège à éviter

Partager la même base de données entre tests parallèles. Sans isolation, les tests se marchent dessus (un test supprime un lead qu'un autre test attendait). Utiliser une DB par worker Vitest via `DATABASE_URL=file:./db/test-${process.env.VITEST_WORKER_ID}.db`.

Piège à éviter

Mocker Prisma systématiquement dans les tests unitaires. Les mocks découplent les tests de la base, mais ils introduisent un risque : si le schéma change, le mock ne reflète pas le changement et le test passe alors que le code est cassé. Privilégier les tests d'intégration avec une vraie DB SQLite in-memory.

Piège à éviter

Lancer Playwright sans `workers` : 1. Plusieurs workers partagent le même serveur et la même DB — les tests E2E se marchent dessus. Soit sérialiser, soit donner une DB et un port distincts par worker.

9. R-010 — Headers de sécurité HTTP

9.1 Objectif

Compléter les en-têtes de sécurité HTTP du middleware. La version actuelle n'envoie que X-Frame-Options, X-Content-Type-Options et Referrer-Policy. Il manque quatre en-têtes critiques : Content-Security-Policy (anti-XSS), Strict-Transport-Security (force HTTPS), Permissions-Policy (limite les API navigateur) et X-DNS-Prefetch-Control.

9.2 Code actuel problématique

src/middleware.ts · TypeScript

```
// src/middleware.ts (extrait)
export function middleware(req: NextRequest) {
  const res = NextResponse.next();
  res.headers.set("X-Frame-Options", "DENY");
  res.headers.set("X-Content-Type-Options", "nosniff");
  res.headers.set("Referrer-Policy", "strict-origin-when-cross-origin");
  // ← manquent : CSP, HSTS, Permissions-Policy, X-DNS-Prefetch-Control
  return res;
}
```

9.3 Helper — securityHeaders

```
// src/lib/security-headers.ts (partie 1/2)
import { NextResponse } from "next/server";

/**
 * Politique CSP stricte mais compatible Next.js.
 * - default-src 'self' : seules les ressources du même domaine
 * - script-src : 'self' + unsafe-inline (Next.js le nécessite en dev)
 * + nonce dynamique en production (voir 9.4)
 * - img-src : autorise LinkedIn CDN et avatars GitHub
 * - connect-src : autorise API backend + webhook LinkedIn
 */
export function buildCsp(nonce?: string): string {
  const isDev = process.env.NODE_ENV === "development";
  const scriptSrc = [
    "'self'",
    isDev ? "'unsafe-inline'" : nonce ? `nonce-${nonce}` : "'unsafe-inline'",
    "'unsafe-eval'", // ← nécessaire pour Next.js dev tools
  ].join(" ");

  return [
    "default-src 'self'",
    `script-src ${scriptSrc}`,
    "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com",
    "font-src 'self' https://fonts.gstatic.com",
    "img-src 'self' data: blob: https: https://media.licdn.com",
    "https://avatars.githubusercontent.com",
    "connect-src 'self' https://api.linkedin.com https://*.upstash.io",
    "frame-ancestors 'none'",
    "base-uri 'self'",
    "form-action 'self'",
    "object-src 'none'",
    "upgrade-insecure-requests",
  ].join("; ");
}
```

```
// src/lib/security-headers.ts (partie 2/2)
export const SECURITY_HEADERS: Record<string, string> = {
  "X-Frame-Options": "DENY",
  "X-Content-Type-Options": "nosniff",
  "Referrer-Policy": "strict-origin-when-cross-origin",
  "Strict-Transport-Security":
    "max-age=63072000; includeSubDomains; preload",
  "Permissions-Policy":
    "camera=(), microphone=(), geolocation=(), browsing-topics=()",
  "X-DNS-Prefetch-Control": "off",
  "Cross-Origin-Opener-Policy": "same-origin",
  "Cross-Origin-Resource-Policy": "same-origin",
};

export function applySecurityHeaders(
  res: NextResponse,
  nonce?: string,
): NextResponse {
  for (const [k, v] of Object.entries(SECURITY_HEADERS)) {
    res.headers.set(k, v);
  }
  res.headers.set("Content-Security-Policy", buildCsp(nonce));
  return res;
}
```

9.4 Intégration dans le middleware

```
// src/middleware.ts (réécrit)
import { NextRequest, NextResponse } from "next/server";
import { randomBytes } from "crypto";
import { applySecurityHeaders } from "@lib/security-headers";

export function middleware(req: NextRequest) {
  // Génère un nonce par requête pour CSP
  const nonce = randomBytes(16).toString("base64");

  const res = NextResponse.next({
    request: {
      headers: new Headers(req.headers),
    },
  });

  // Injecte le nonce dans la requête pour que Next.js l'utilise dans <script>
  res.headers.set("x-nonce", nonce);

  return applySecurityHeaders(res, nonce);
}

export const config = {
  matcher: [
    "/((?!_next/static|_next/image|favicon.ico|robots.txt).*)",
  ],
};
```

9.5 Test des en-têtes

Valider les en-têtes via un test automatisé et via un scanner externe. Le test local vérifie la présence et la valeur de chaque en-tête. Le scanner externe (securityheaders.com, observador.mozilla.org) fournit une note objective et révèle les oublis.

tests/integration/security-headers.test.ts · TypeScript

```
// tests/integration/security-headers.test.ts
import { describe, it, expect } from "vitest";

const BASE = process.env.TEST_API_URL ?? "http://localhost:3000";

describe("Headers de sécurité", () => {
  it("présents sur toutes les routes HTML", async () => {
    const res = await fetch(`${BASE}/`);
    expect(res.headers.get("x-frame-options")).toBe("DENY");
    expect(res.headers.get("x-content-type-options")).toBe("nosniff");
    expect(res.headers.get("referrer-policy"))
      .toBe("strict-origin-when-cross-origin");
    expect(res.headers.get("strict-transport-security"))
      .toContain("max-age=63072000");
    expect(res.headers.get("permissions-policy"))
      .toContain("camera=()");
    expect(res.headers.get("content-security-policy"))
      .toContain("default-src 'self'");
  });

  it("HSTS absent en HTTP dev", async () => {
    if (process.env.NODE_ENV !== "production") return;
    const res = await fetch(`${BASE}/`, { redirect: "manual" });
    expect(res.headers.get("strict-transport-security")).toBeTruthy();
  });
});
```

9.6 Checklist de validation

- ☐ Le header Content-Security-Policy est présent sur toutes les routes HTML
- ☐ Le header Strict-Transport-Security est actif en HTTPS production
- ☐ Le header Permissions-Policy désactive camera, micro, géoloc
- ☐ Le scanner securityheaders.com donne une note A ou A+
- ☐ Le scanner observador.mozilla.org ne signale aucun warning
- ☐ Aucune erreur CSP dans la console navigateur sur les pages critiques
- ☐ Les scripts inline utilisent un nonce dynamique (pas `unsafe-inline` en prod)

9.7 Pièges à éviter

Piège à éviter

Activer CSP `default-src 'none'` d'un coup. Cette politique bloque tout (CSS, images, fonts) et casse immédiatement le rendu. Toujours commencer par `default-src 'self'` et assouplir au cas par cas. Utiliser le mode `Report-Only` d'abord pour collecter les violations sans bloquer.

Piège à éviter

Oublier d'autoriser les domaines externes réellement utilisés (LinkedIn CDN, Google Fonts, avatars GitHub). La CSP bloque silencieusement ces ressources — les images ne s'affichent pas, les fonts tombent en fallback. Auditer le HTML rendu pour lister les domaines référencés.

Piège à éviter

Garder `unsafe-inline` en production par commodité. Cela annule l'efficacité de la CSP contre les attaques XSS. Migrer vers un nonce dynamique ou un hash SHA par script. Next.js 16 supporte nativement les nonces via `headers()` dans le middleware.

10. Quick wins P2 — Optimisations secondaires

Ce chapitre regroupe cinq correctifs P2 exécutables en moins d'une demi-journée chacun. Ils ne sont pas bloquants pour la mise en production mais améliorent significativement la qualité, la performance ou la maintenabilité. Ils peuvent être traités en parallèle des chantiers P0/P1 ou en intercalaire.

10.1 Optimisation des images

Le flag `images.unoptimized = true` désactive l'optimisation Next.js (conversion WebP/AVIF, redimensionnement, lazy-loading). Le corriger permet de réduire la taille des images transmises de 60 à 80 % selon les formats sources.

Avant

```
// AVANT — next.config.ts
images: {
  unoptimized: true,
},
```

Après

```
// APRÈS — next.config.ts
images: {
  formats: ["image/avif", "image/webp"],
  deviceSizes: [640, 750, 828, 1080, 1200, 1920],
  imageSizes: [16, 32, 48, 64, 96, 128, 256, 384],
  remotePatterns: [
    { protocol: "https", hostname: "media.licdn.com" },
    { protocol: "https", hostname: "avatars.githubusercontent.com" },
    { protocol: "https", hostname: "lh3.googleusercontent.com" },
  ],
  minimumCacheTTL: 60 * 60 * 24, // 24 h
}
```

10.2 Unifier les fichiers de déploiement

Le projet contient cinq fichiers de configuration de déploiement (`render.yaml`, `netlify.toml`, `Caddyfile`, `docker-compose.yml`, `build.sh`). Cette dispersion crée de la confusion et un risque de divergence. Choisir UNE plateforme cible (recommandé : Render pour la simplicité, ou Vercel pour l'intégration native Next.js) et supprimer les autres fichiers.

Fichier	Plateforme	Action	Risque
render.yaml	Render	Conserver	Aucun
netlify.toml	Netlify	Supprimer	Aucun
Caddyfile	Reverse proxy	Supprimer	Aucun si Render gère le proxy
docker-compose.yml	Local / K8s	Conserver pour dev local	Aucun

Fichier	Plateforme	Action	Risque
build.sh	Build script	Migrer vers package.json	Faible

10.3 Logger structuré

Remplacer les `console.log` et `console.error` épars dans le code par un logger structuré compatible JSON. Cela facilite l'ingestion par Datadog, ELK ou Logflare et permet le filtrage par `correlationId`, `userId`, `level`.

```
// src/lib/logger.ts
type LogLevel = "debug" | "info" | "warn" | "error";

interface LogEntry {
  level: LogLevel;
  message: string;
  timestamp: string;
  [key: string]: unknown;
}

const LEVEL_PRIORITY: Record<LogLevel, number> = {
  debug: 10, info: 20, warn: 30, error: 40,
};

const MIN_LEVEL: LogLevel =
  (process.env.LOG_LEVEL as LogLevel) ??
  (process.env.NODE_ENV === "production" ? "info" : "debug");

export const logger = {
  debug(msg: string, ctx?: Record<string, unknown>) { log("debug", msg, ctx); },
  info(msg: string, ctx?: Record<string, unknown>) { log("info", msg, ctx); },
  warn(msg: string, ctx?: Record<string, unknown>) { log("warn", msg, ctx); },
  error(msg: string, ctx?: Record<string, unknown>) { log("error", msg, ctx); },
};

function log(level: LogLevel, message: string, ctx?: Record<string, unknown>) {
  if (LEVEL_PRIORITY[level] < LEVEL_PRIORITY[MIN_LEVEL]) return;

  const entry: LogEntry = {
    level,
    message,
    timestamp: new Date().toISOString(),
    ...ctx,
  };

  const out = process.env.NODE_ENV === "production"
    ? JSON.stringify(entry)
    : `[${entry.timestamp}] ${level.toUpperCase()} ${message}\n${
      ctx ? JSON.stringify(ctx, null, 2) : ""
    }`;

  const stream = level === "error" ? process.stderr : process.stdout;
  stream.write(out + "\n");
}
```

10.4 Variables d'environnement typées

Centraliser la lecture des variables d'environnement via un helper typé. Cela évite les `process.env.X!` éparpillés, garantit la validation au démarrage et facilite l'inventaire des variables requises.

```
// src/lib/env.ts
import { z } from "zod";

const EnvSchema = z.object({
  NODE_ENV: z.enum(["development", "test", "production"]),
  DATABASE_URL: z.string().min(1),
  NEXTAUTH_SECRET: z.string().min(16),
  NEXTAUTH_URL: z.string().url(),
  GITHUB_CLIENT_ID: z.string().min(1),
  GITHUB_CLIENT_SECRET: z.string().min(1),
  UPSTASH_REDIS_URL: z.string().url(),
  UPSTASH_REDIS_TOKEN: z.string().min(1),
  OPENAI_API_KEY: z.string().min(1).optional(),
  LINKEDIN_CLIENT_ID: z.string().min(1).optional(),
  LINKEDIN_CLIENT_SECRET: z.string().min(1).optional(),
  SENTRY_DSN: z.string().url().optional(),
});

const parsed = EnvSchema.safeParse(process.env);

if (!parsed.success) {
  console.error("Variables d'environnement invalides:");
  for (const issue of parsed.error.issues) {
    console.error(` - ${issue.path.join(".")}: ${issue.message}`);
  }
  process.exit(1);
}

export const env = parsed.data;
export type Env = typeof env;
```

10.5 Lint rules personnalisés

Ajouter un lint rule personnalisé qui bloque l'introduction de `any` et de `console.log`. Ce garde-fou empêche la réintroduction de la dette technique corrigée au chapitre 4.

.eslintrc.additional.js · JavaScript

```
// .eslintrc.additional.js
module.exports = {
  rules: {
    "@typescript-eslint/no-explicit-any": "error",
    "no-console": ["error", { allow: ["warn", "error"] }],
    "@typescript-eslint/consistent-type-imports": "warn",
    "prefer-const": "error",
    "no-unused-vars": ["error", { argsIgnorePattern: "^_" }],
  },
  overrides: [
    {
      files: ["scripts/**", "prisma/**"],
      rules: { "no-console": "off" },
    },
  ],
};
```

10.6 Checklist globale P2

- ☐ `images.unoptimized` est supprimé de `next.config.ts`
- ☐ Le nombre de fichiers de déploiement est réduit à 2 maximum
- ☐ Le logger structuré remplace les `console.log`
- ☐ Le fichier `src/lib/env.ts` valide toutes les variables au boot
- ☐ Les lint rules custom sont actifs et bloquent les nouveaux `any`
- ☐ Les images servies sont en WebP/AVIF (vérifier l'en-tête `Content-Type`)

11. Conclusion et prochaines étapes

Ce Volume 2 a transformé les dix-neuf recommandations stratégiques du Volume 1 en un ensemble de livrables techniques directement actionnables. Chacun des huit chapitres P0/P1 fournit le code corrigé, les helpers réutilisables, les checklists de validation et la liste des pièges connus. Le développeur en charge d'un risque peut désormais livrer la correction sans avoir à reconcevoir la solution, et sans risque de réintroduire un défaut déjà identifié.

Les dix helpers livrés — `hashPassword`, `requireSession`, `requireUser`, `assertOwnership`, `narrowUnknown`, `prismaLog`, `rateLimiters`, `ApiError`, `withErrorHandler` et `securityHeaders` — constituent une boîte à outils cohérente qui doit être déposée dans `src/lib/` avant le démarrage des chantiers. Ils forment le socle technique transverse du projet et faciliteront tout refactor futur.

Recommandation finale

Traitement dans l'ordre : P0 (chapitres 2 à 4) en semaines 1 à 2, P1 (chapitres 5 à 9) en semaines 3 à 8, quick wins P2 (chapitre 10) en intercalaire dès qu'une fenêtre de une demi-journée se présente. Ne pas paralléliser P0 et P1 — les P1 dépendent structurellement des P0 (par exemple `requireUser()` suppose que `requireSession()` existe). Une fois les P0 livrés et stabilisés en staging, entamer les P1.

11.1 Prochaines étapes recommandées

- **Semaine 1** — Déposer les 10 helpers dans `src/lib/` via une PR dédiée, sans modifier les routes existantes. Cela permet de valider la compilation et le typage avant tout chantier.
- **Semaines 2 à 3** — Traiter R-001 (auth) et R-002 (multi-tenant) ensemble, car ils sont structurellement liés. Une PR unique par couple (helper + routes migrées) facilite la review.
- **Semaines 4 à 5** — Traiter R-003 (build strict) et R-004/005 (base de données). Ces deux chantiers ne touchent pas les mêmes fichiers et peuvent être parallélisés si deux développeurs sont disponibles.
- **Semaines 6 à 8** — Traiter R-007 (rate-limit), R-008 (erreurs API) et R-010 (headers sécurité). Ces trois sujets sont indépendants et peuvent être menés en parallèle.
- **Semaines 9 à 11** — Construire la stratégie de test (R-009). Attendre que les P0/P1 soient livrés pour éviter d'écrire des tests sur du code en cours de refactoring.
- **Semaine 12** — Quick wins P2 (chapitre 10) et préparation du Volume 3 dédié aux optimisations de second ordre (P2 restants).

11.2 Métriques de succès

Métrique	Valeur initiale	Cible S2	Cible S8	Cible S12
Nombre de any	87	60	20	5

Métrique	Valeur initiale	Cible S2	Cible S8	Cible S12
Couverture src/lib	0%	30%	60%	70%
Couverture src/app/api	0%	10%	40%	50%
Routes API wrappées withErrorHandler	0%	50%	100%	100%
Routes multi-tenant (requireUser)	0%	50%	100%	100%
Note securityheaders.com	F	B	A	A+
Erreurs 500 / semaine (Sentry)	N/A	<50	<10	<5

Tableau — Métriques mesurables pour valider la progression de la roadmap.

11.3 Liens avec le Volume 3

Le Volume 3, à publier après livraison du présent volume, traitera les neuf risques P2 restants : nettoyage des **any** résiduels, configuration fine des images par route, suppression des caches in-memory résiduels, audit de l'orchestrateur de workflows, optimisation du bundle client, accessibility (a11y), internationalisation (i18n), documentation API OpenAPI et stratégie de migration progressive vers PostgreSQL. Il inclura également le retour d'expérience post-implémentation du Volume 2 et les ajustements éventuels.